

Reproducible Build — pqc_crypto_module v0.1.0

Disclaimer: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

1. Purpose

This document describes how to produce a reproducible build of pqc_crypto_module for FIPS 140-3 integrity verification. A reproducible build ensures that the same source code, toolchain, and dependencies produce a bit-identical binary.

2. Toolchain

Component	Version	Source
Rust compiler	Nightly (workspace requirement)	rustup
Cargo	Bundled with Rust	rustup
Target	Platform-native (e.g., x86_64-unknown-linux-gnu, aarch64-apple-darwin)	rustup target list

The exact Rust nightly version used for a given build should be recorded. Check with:

```
rustc --version --verbose
```

Pin the toolchain version in rust-toolchain.toml at the workspace root for reproducibility:

```
[toolchain]
channel = "nightly-2025-04-01" # Example; use the actual pinned
                                date
```

3. Dependency Pinning

All dependency versions are pinned via `Cargo.lock` at the workspace root. This file must be committed to version control and must not be modified between builds intended to be reproducible.

Key dependencies and their pinned versions (from `Cargo.toml`):

Crate	Version Spec	Purpose
pqcrypto-mldsa	0.1.2	ML-DSA-65
pqcrypto-traits	0.3	PQC type traits
sha3	0.10	SHA3-256
sha2	0.10	SHA-256 (legacy)
hmac	0.12	HMAC (legacy)
ed25519-dalek	2.1	Ed25519 (legacy)
rand	0.8	CSPRNG
rand_core	0.6	getrandom backend
zeroize	1.7	Memory zeroization
thiserror	1.0	Error derivation
hex	0.4	Hex encoding

To verify dependency integrity:

```
cargo verify-project
cargo tree -p pqc_crypto_module
```

4. Build Commands

Standard build

```
cargo build -p pqc_crypto_module --release
```

Approved-only build (no legacy algorithms)

```
cargo build -p pqc_crypto_module --release --features approved-only
```

Verification build

```
# Clean and rebuild from scratch
cargo clean -p pqc_crypto_module
cargo build -p pqc_crypto_module --release

# Record the hash of the output
sha256sum target/release/libpqc_crypto_module.rlib
```

5. Deterministic Build Considerations

Rust builds are not fully deterministic by default due to:

Factor	Mitigation
Timestamps	Rust does not embed timestamps in <code>.rlib</code> files
File ordering	Cargo compiles in deterministic order
Path dependencies	Use <code>CARGO_HOME</code> in a fixed location
Randomized hashing	Compiler uses deterministic hashing for builds
C dependencies	<code>pqcrypto-mlds</code> links to C reference implementations; ensure the same C compiler and flags

For maximum reproducibility:

```
# Fix the build directory
export CARGO_TARGET_DIR=/tmp/pqc_build

# Disable incremental compilation
export CARGO_INCREMENTAL=0

# Use a fixed CARGO_HOME
export CARGO_HOME=/tmp/cargo_home

# Build
cargo build -p pqc_crypto_module --release
```

6. Source Integrity

The module source is versioned in Git. To verify source integrity:

```
# Record the commit hash
git rev-parse HEAD

# Verify no uncommitted changes
git status --porcelain

# List all files in the cryptographic boundary
ls -la crates/pqc_crypto_module/src/
```

The source files constituting the cryptographic boundary are:

```
crates/pqc_crypto_module/src/
  api.rs
  approved_mode.rs
  errors.rs
  hashing.rs
  legacy.rs
```

```
lib.rs
mldsa.rs
mlkem.rs
rng.rs
self_tests.rs
types.rs
```

7. Build Artifact Identification

After building, the primary artifact is:

```
target/release/libpqcrypto_module.rlib
```

Record the following for each build:

- Git commit hash
- Rust compiler version (`rustc --version --verbose`)
- Target triple
- SHA-256 hash of the `.rlib` file
- Feature flags used (default or `approved-only`)
- Build date

8. Continuous Integration

The CI pipeline should:

1. Pin the Rust nightly version.
2. Use `Cargo.lock` without modification.
3. Run `cargo fmt --check` and `cargo clippy -- -D warnings`.
4. Run all tests: `cargo test -p pqcrypto_module`.
5. Build in release mode and record the artifact hash.
6. Archive the build log, compiler version, and artifact hash.